

# How Hardware Gets Hacked (Part 6)

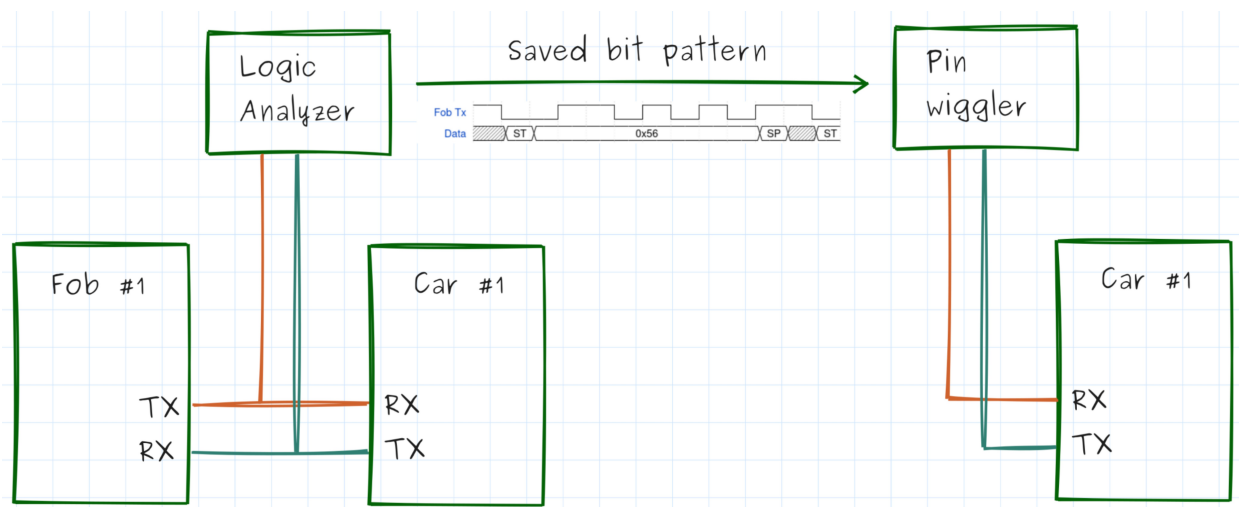
## Authenticated Exchanges

by Nathan Jones

|                                              |    |
|----------------------------------------------|----|
| Introduction.....                            | 1  |
| Defense #2b: Rolling counters with MACs..... | 2  |
| Updated threat model.....                    | 8  |
| Industry threat models.....                  | 10 |
| Conclusion.....                              | 12 |

## Introduction

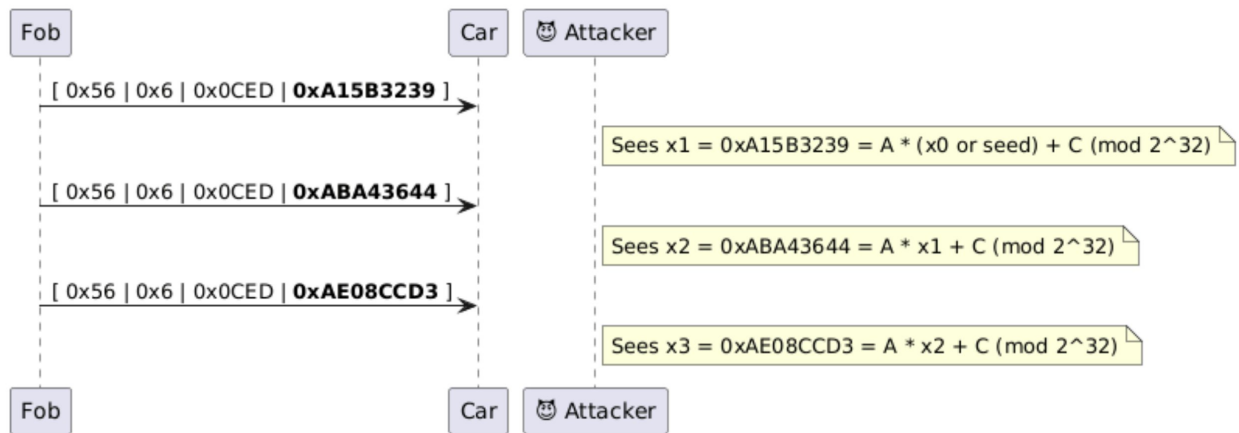
The [last article](#) left off on a bit of a cliff-hanger: an attacker can easily conduct a replay attack on our car and we have no defense against it yet!



We determined in that article that, ultimately, we need a defense that ensures:

- The car and any paired fobs share a “secret” only they know
- This secret is not something that is revealed during unlocking, given the adversarial nature of our communication channel
- Messages change every time a fob tries to unlock

The second bullet is possibly the hardest to achieve. We tried last time to use a PRNG (pseudo-random number generator) and discovered that it's easily attacked if a person observes just three consecutive messages.



What we need is math operation that won't reveal its inputs even if an attacker can see all of the outputs: a "one-way" operation, a.k.a. a **cryptographically secure** math operation. In this article, we'll see that math operation in action and finally close off these replay attacks! We'll spend a bit of time discussing the nuance in this defense and then update our threat model. We'll also briefly discuss industry approaches threat modeling.

## Defense #2b: Rolling counters with MACs

Given that it seems like we're looking for a "cryptographically secure" version of our PRNG, you might think at this point that we're going to turn to a cryptographically secure PRNG (CSPRNG) to generate our random numbers. (Yes, they exist! A CSPRNG generates pseud-random numbers, but in a way that's impossible for anyone who sees the generated numbers to prove what the inputs must have been.) Unfortunately we'd run into a small problem when we tried to pair new fobs. The reason is because, in the 2023 eCTF, pairing a new fob only required a paired fob and an unpaired fob; the car was nowhere in the process.

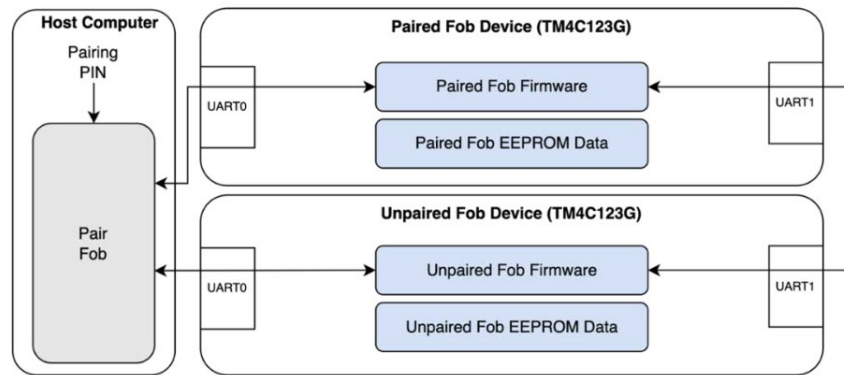
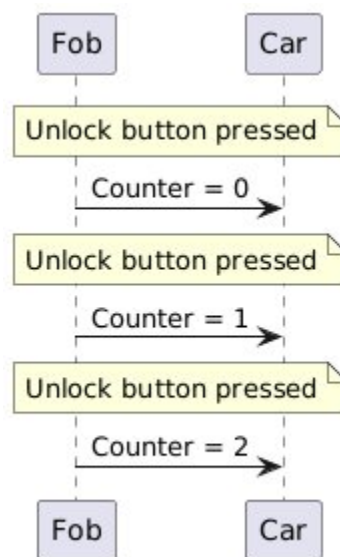


Figure 6. Pair Fob Step

Figure 3: 2023 eCTF Rules, <https://ectf.mitre.org/wp-content/uploads/2023/02/2023-eCTF-Rules-v1.1-lr.pdf>

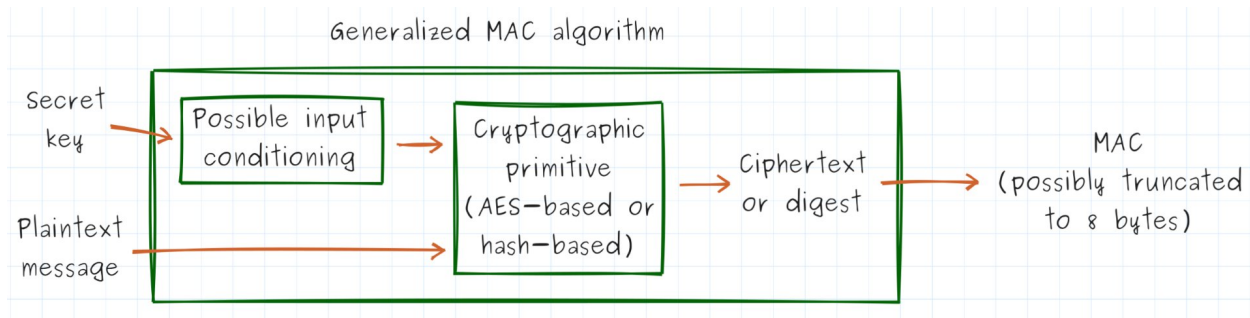
If a newly paired fob tried to unlock the associated car, the car would have no knowledge of the fob or its proper sequence of pseudo-random numbers. (This is a slight deviation from most production systems, which typically ensure that the car is part of the pairing process so that it knows, unequivocally, which fobs are paired and which aren't.)

This presents a bit of a “chicken and egg” problem and the solution is to replace the PRNG sequence with a **rolling counter**. Literally, each fob that is paired to a car sends a 0 in its first unlock message, followed by a 1, then a 2, etc; no pseudorandomness at all. There's no need for the car to have knowledge of a fob's number sequence if it's assumed that it always starts at 0 and merely increments on each unlock message.



This is, of course, ridiculously insecure so to complete the design we need to add our cryptographically secure mathematical operation, something that can take each counter value and a “secret” as input and produce an output that, to anybody without the “secret”,

looks like total randomness. The operations that have this property are few and well-studied; not something your ordinary engineer is going to successfully create on their own. A few examples of these special operations are **HMAC-SHA256**, **Poly1305**, **AES-CMAC**, and **AES-GCM**. Each of them takes in a secret key and our message and uses a cryptographically secure function like AES or SHA256 to produce an output that can't be used to reverse the secret; this output is often then truncated to 8 bytes.



This output value is known as a **message authentication code (MAC)** since the only way it could be produced correctly for a given message is if the sender knew the secret key; in essence, a **valid MAC authenticates an unlock message**.

Our defense, then, works like this:

At build-time, a random number (the secret key) is generated for each car and baked into the firmware for that car and each paired fob. The fob also gets an ID (can be random or incrementing from 0).

```

// car_gen_secret.py
# Get key for car ID if already made, otherwise:
key_array = list(random.randbytes(16))
secrets["keys"][args.car_id] = key_array

// fob_gen_secret.py
fp.write(f'#define FOB_ID {fob_id}\n')
# If paired:
fp.write('#define KEY {')
for i in range(15):
    fp.write(f'{key_array[i]}, ')
fp.write(f'{key_array[15]}'}\n')
  
```

These each produce a `secrets.h` file like the ones below.

```

// Example snippet from secrets.h for a car
#define KEY {102, 156, 198, 252, 111, 199, 160, 85,
  
```

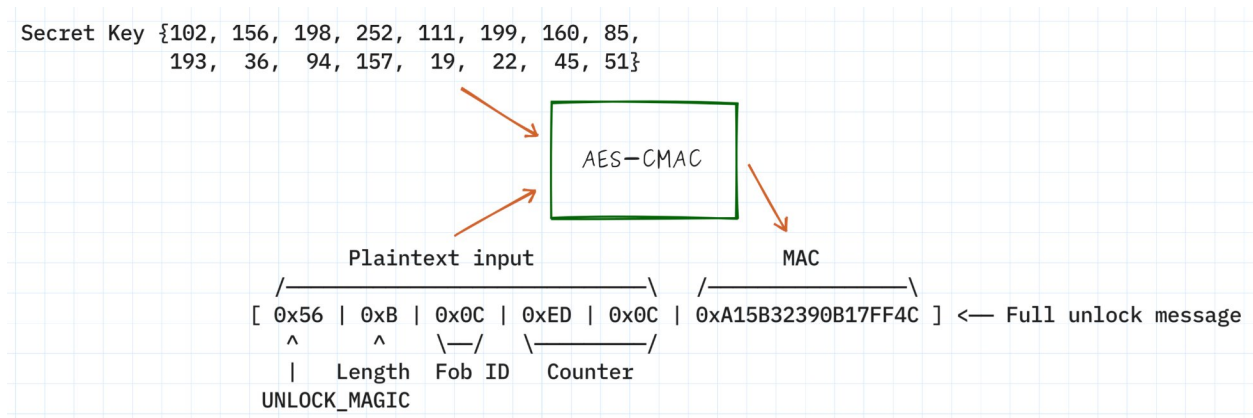
```

193, 36, 94, 157, 19, 22, 45, 51}

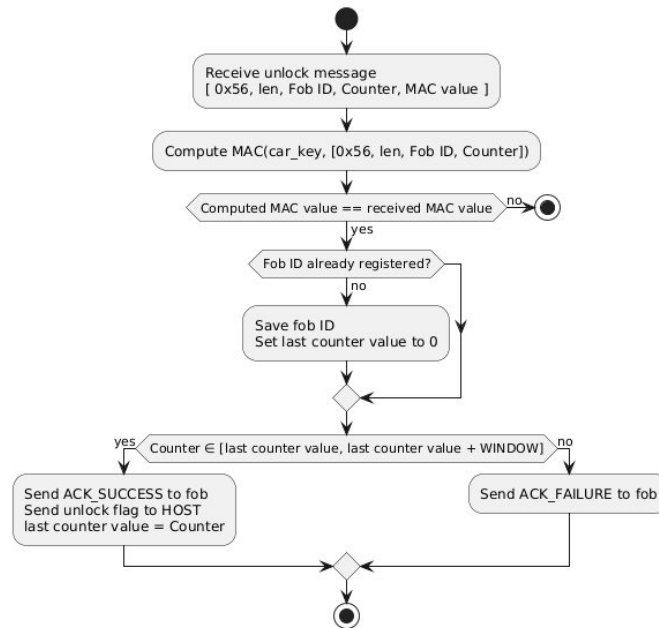
// Example snippet from secrets.h for this car's paired fob
#define FOB_ID 139
#define KEY {102, 156, 198, 252, 111, 199, 160, 85,
            193, 36, 94, 157, 19, 22, 45, 51}

```

To unlock a car, the fob sends an unlock message built from its fob ID, its next counter value (one more than the last counter value sent), and the MAC value computed over the whole message, e.g.



I'll be using the [tiny-AES-CMAC-c](#) and [tiny-AES-c](#) libraries to do this. (Mbed TLS and WolfSSL are other popular cryptography libraries; they are likely better vetted but may also not be size-optimized for embedded systems, especially when all we need at this point is message authentication.)



When the car receives an unlock message, it checks that it gets the same MAC value when computed over the entire message. If so, it then gets the last valid counter value it received from that fob; if it has no record of the fob ID, it assumes that the fob has been newly paired and creates a new spot in memory to store its counter values, starting at 0.

If the received counter value is between the last counter value and that value plus a small window, the car updates the last seen value to the value just received and proceeds to unlock, sending ACK SUCCESS to the fob. If the counter value is not in range, the car fails to unlock and sends back the ACK FAILURE message. This rolling window helps prevent simple replay attacks like re-transmitting the unlock message that was just sent, since the car will already have moved on to the next counter value.

It's also important that the car first updates its internal counter and *then* transmits any flags, ***in that exact order***, since an attacker who is able to reset the car after a SUCCESS message is sent but before the car commits the new counter to memory has a valid unlock message that the fob just sent, effectively defeating our rolling counter scheme.

Does this approach now satisfy our requirements?

- The attacker never sees or hears the “secret” (i.e. the random car key generated at build-time), even if they were watching every unlock message sent by a fob to a car
- The car rejects old codes, preventing simple replay attacks
- The only way a person or device could determine the MAC value for a given message (proven by cryptographic researchers for the operations above) is to know that car key generated at build-time, which is only known to the fob and car.

Yes, it seems like it does! More than that, it passes the security tests I wrote!

```
(hghVenv) nathancharlesjones@GeorgeWashingtonCarver:~/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code$ \
> pytest testing/security_tests.py
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code
collected 2 items

testing/security_tests.py .. [100%]

===== 2 passed in 1.66s =====
```

—[? BRAINSTORM]—

### Are you satisfied?

Does this constitute proof that I've closed off the attack vector described above? What else, if anything, would you need to be confident that this defense does what it's intended to do? Jot down your thoughts before continuing on.

Interestingly, this is almost the exact same solution that KeeLoq arrived at in the late-1980s for securing production key fobs (the only difference being that KeeLoq's system *encrypted* the rolling counter instead of just computing the MAC, which was a design choice that made sense for the time).

One last thing to note is that our very simple implementation papers-over a few real-life design considerations we would need to make for a production system. For instance:

- **Inequality check:** Checking that the counter value the car received is within range of the approved WINDOW requires a careful construction to ensure that it still works when the upper bound and/or the actual counter roll over to 0 (which I've done in the example code).
- **WINDOW size:** The size of the WINDOW needs to be selected so as to not be so small as to annoy the user, but not so large as to give attackers more room to attempt an attack.
- **Resynchronization:** Even still, a fob may get triggered enough times to exceed the WINDOW. In a production system, there would need to be a way for the two devices to “resync” with each other, probably preferably without having to take the car to a dealership.
- **Maximum number of fobs that can pair with a car:** A production car will only be able to hold a certain number of fobs in memory, so checking if a fob has been paired would be slightly more complex than simply looking up a value in an array (we've skirted this issue by capping the number of possible fobs, company-wide, to

256 so fob IDs only need to be a byte and a single car can conceivably store counter values for each one). A car may also need to reject a pairing if it's memory is full.

- **Deregistering a fob:** A real system would need to somehow provide the capability to “deregister” a fob from a car.
- **Flash wearout:** Flash memory typically fails after 10-100k write cycles and our system writes to flash every time the fob tries to unlock a car (or is paired or has a feature enabled) or anytime the car is successfully unlocked. If a person unlocks their car three times a day, then either device may fail within 10 years, which is a very reasonable lifespan for a fob. A robust design should handle possible memory failure by identifying worn out sections of flash and moving saved data to a different place if detected. The system could also do this proactively by writing saved data to a different spot in flash on each write, in a process called “wear leveling”.
- **MAC failure or counter out-of-range:** What should the device do if an unlock message fails either the MAC check or the rolling counter check? Our system simply rejects the unlock request, but we may also want to consider being a bit more proactive in our response. Some options (in order of increasing severity) are to:
  - o **Increment a counter** of failed attempts (for use in debugging later)
  - o **Enforce a cool-down period** after X unsuccessful attempts in which the car doesn't react to any unlock message for Y seconds or Z minutes.
  - o **Lock down the device**, refusing all unlock attempts until a more rigorous form of authentication is given (a dealer entering a special code, a user entering the fob pin, etc).

—[? BRAINSTORM]—

**“There, I fixed it.”**

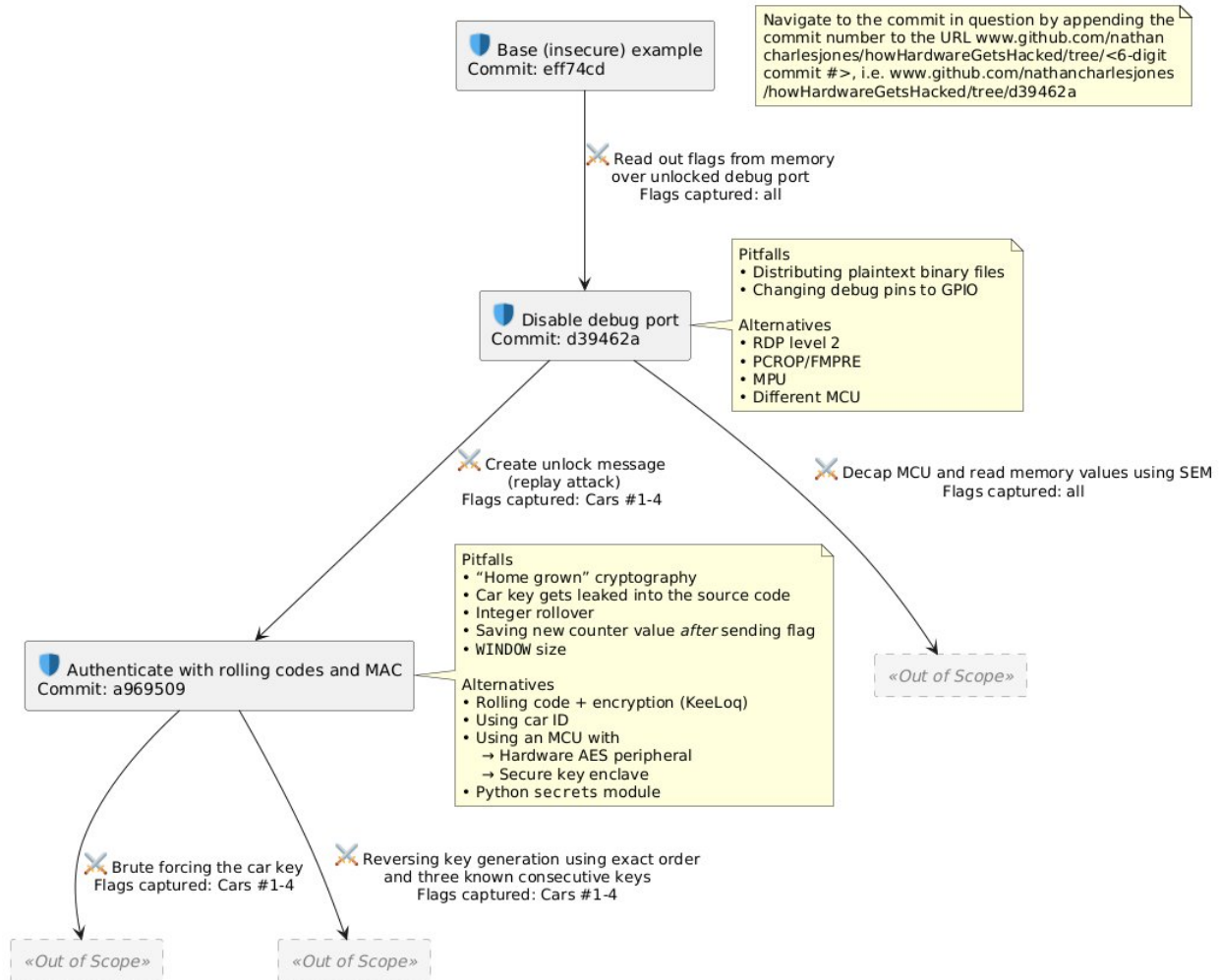
Using a rolling counter and MAC doesn't just close off that attack path, it might actually open up new ones! Take a minute to brainstorm how an attacker might get around our defense or what other attacks might now be possible.

---

## Updated threat model

Lets use this new attack and defense to update our threat model, specifically our depiction of “Attacks and Defenses”; there has been no change to the sections “Attackers” or “Secrets/Payoffs”.





As mentioned above, there are several pitfalls to avoid when implementing this defense.

- Don't use a MAC algorithm that hasn't been verified as cryptographically secure (as HMAC-SHA256, Poly1305, AES-CMAC, and AES-GCM are). **DON'T TRY TO WRITE YOUR OWN CRYPTOGRAPHIC ROUTINE, NO MATTER HOW SIMPLE AND COMPLETE IT MAY SEEM.**
- Ensure that you haven't accidentally exposed the car key in your build scripts.
- Ensure that the inequality check comparing the received rolling counter to the one in memory properly handles integer rollover.
- Ensure that the car correctly updates its internal counter *before* sending out its flags.
- Avoid making the WINDOW size too high (allows an attacker more room to try and brute force an attack) or too small (users will become annoyed if they are locked out of their car because they accidentally pressed the unlock button too many times while out of range of the car).

A few design alternatives to consider are:

- Using a rolling code + encryption, like KeeLoq did
- Using the car ID:
  - In the fob's message, so that a car has additional verification that the fob is truly trying to unlock it
  - In an initial reply from the car, so that the fob has verification that it's talking to the car to which it's been paired
- Using an MCU with
  - A hardware AES peripheral (runs faster than a software library; possibly better vetted)
  - A "secure enclave" for key storage (can't be read, even if an attacker gains access to the flash; often has a means of zeroizing the key that can be triggered if an attacker opens up the device enclosure)
- Using the `secrets` module to generate keys instead of `random`. : E.g. `key_array = list(secrets.token_bytes(16))` vice `key_array = list(random.randbytes(16))`. The difference is that `random` uses a PRNG and `secrets` uses a CSPRNG. Although this was a big concern for unlocking when our attacker could potentially see three consecutive unlock messages, it doesn't seem feasible to me for an attacker to know the exact order in which keys were generated *and* to then discover three such keys, consecutively generated, in order to use the algebra above to determine the remaining keys.

We can also add a few "out of scope" attacks that we are intentionally not going to defend against.

- Brute forcing the car key by sending a forged unlock message and picking a random value for the MAC. Technically you have a 1 in  $2^{256}$  chance of guessing it right on the first try, but otherwise this is computationally infeasible. There's a reason they call AES "secure".
- An attacker determines the exact order in which each key was generated by the build system for Cars #1-5 and is able to discover the value of three such consecutive keys. They can then use the algebra above to determine the values of every other key.

## Industry threat models

Additionally, I've done some further research since the last article and I think I made an oversimplification when I said that there are "no industry standards" for threat models. Although it's true that there isn't any *de facto* standard, there are many formalized systems that aim to help developers conduct threat assessments and many of them can approach

“de facto” status for specific industries or types of products. The most broadly applicable are the “Threat Assessment and Remediation Analysis” (TARA) developed by MITRE and primarily adopted by the automotive industry (ISO/SAE 21434) and the “Security Target” document that’s part of the Common Criteria standard for IT security products (ISO/IEC 15408). Others include HEAVENS (Healing Vulnerabilities to Enhance Software Security and Safety), EVITA (E-safety Vehicle Intrusion Protected Apps), SAHARA (Security-aware Hazard and Risk Analysis), FMVEA (Failure Mode Vulnerabilities and Effects Analysis), and PASTA (Process for Attack Simulation and Threat Analysis).

In all of them, a design team essentially conducts a “risk assessment” for a given device, which fundamentally lists a device’s:

- Risks and vulnerabilities,
- Mitigation strategies for those vulnerabilities, and
- The residual risk level, after mitigation.

We essentially did as much in [Part 4 of this series](#), when we listed and ranked a few possible attacks and settled on “Read out flags over an unlocked debug port” as the “most critical risk”. Our mitigation for this was to disable the debug port on production devices and the residual risk level was deemed acceptable.

One possible TARA version of our threat model can be found [here](#), which has three major sections:

| Section                     | Purpose / Outcomes                                                                                                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Scope and Assets            | Describe the device and the data, functions, or capabilities whose loss, disclosure, or corruption would harm the system's security goals. |
| Threat Matrix               | List and rank all possible attacks that could result in an attacker getting an asset they shouldn't have.                                  |
| Mitigation / Solution Table | List each strategy that lessens the damage done by a threat. Iterate until all threats have been reduced to acceptable levels.             |

Notice that the same information, more or less, is present here as in the threat model I developed, just in a tabular format as opposed to a graphical one. Additionally, the threat model I’ve been developing shows more of the evolution of the design as opposed to a single snapshot. You could almost imagine that a threat assessment is being conducted for each defense block, at which point the “most critical risk” is specifically addressed and we update our design. And, in truth, even the formal threat assessments are supposed to be iterative, though many are treated like compliance documents. Ideally, designers are

conducting a threat analysis from the beginning of a project and iterating on that design anytime a threat pops up that's unmitigated or that has a residual risk level that's too high. Each threat analysis can only really be conducted against a certain, specific version of the design, so a re-analysis would also be triggered anytime the design changes, even for reasons that seem unrelated to security.

Critical to the threat modeling process is being able to identify and describe all of the threats/risks to your device, which is tough to do if you're not a security researcher. Security consultants can help in this regard, and a number of methodologies exist to help teams enumerate these threats such as JIL, [STRIDE](#), [EMB3D](#), [ATT&CK](#), CWSS, and CVSS. Only JIL and EMB3D give much attention to hardware-level attacks, though.

I'm partial to the threat model that I've been developing (can you blame me?!), so I'll continue in this fashion. But know that in a more professional setting something like TARA may be more appropriate for your project (more information about TARA can be found [here](#) and [here](#)). Additionally, you can use tools like JIL and [EMB3D](#) to help make sure you're accounting for all plausible threats when you do your threat modeling.

## Conclusion

—[? REFLECTION]—

**What are two things you want to remember about this article?**

- 1.
- 2.

Write them down or say them to yourself in your head before you move on.

---

A PRNG wasn't sufficient to prevent replay attacks exactly because it's a normal, algebraic math operation. An attacker can still solve for the unknowns (i.e. uncover our secrets!) by simply seeing enough outputs.

One solution to this is to use a cryptographically secure algorithm like HMAC-SHA256 or AES-CMAC to produce a "message authentication code" (MAC) for each unlock message, which also now includes a rolling counter. The key used by the algorithm computing the MAC is the "secret" that both car and fob share and which isn't revealed in the unlocking process since algorithms like AES-CMAC are truly "one-way" operations. The counter ensures that the MAC value changes from message to message.

This solution is able to defend against the three attacks discussed at the beginning of the last article. However, there are several pitfalls to avoid in the design of this solution (such as committing updated counter values to flash *before* a car sends the unlock flag to the host) and several considerations that production-level systems would need to make (such as handling resynchronization or failed unlock attempts).

If you've made it this far, thanks for reading and happy hacking!